

Business Management System (BMS) - Development Documentation

Table of Contents

1. [System Overview](#)
 2. [Architecture](#)
 3. [Technology Stack](#)
 4. [Project Structure](#)
 5. [Core Features](#)
 6. [API Documentation](#)
 7. [Authentication & Authorization](#)
 8. [Database Schema](#)
 9. [File Handling](#)
 10. [Frontend Structure](#)
 11. [Setup & Installation](#)
 12. [Key Workflows](#)
 13. [Environment Variables](#)
 14. [Common Issues & Solutions](#)
-

System Overview

The Business Management System (BMS) is a comprehensive web application designed to manage business operations, compliance, KYC processes, and client relationships. The system supports multiple user roles with specific permissions and provides workflow management for various business processes.

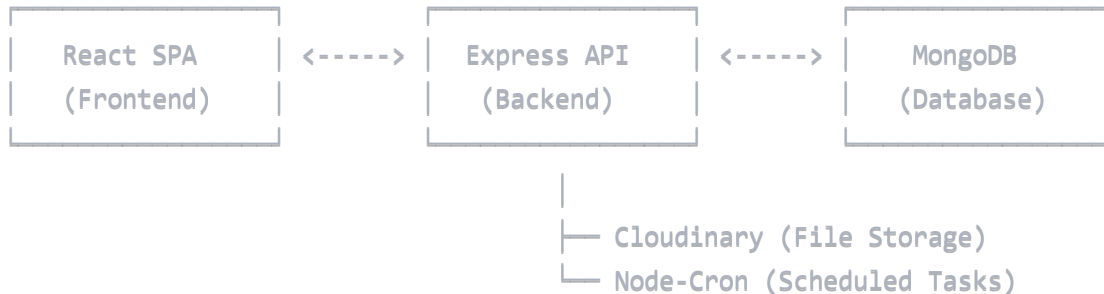
Key Capabilities:

- Job creation and lifecycle management
- Multi-stage approval workflows (Compliance, KYC, BRA)
- Client and company information management
- Document management and storage
- Payment tracking and invoicing
- Automated notifications and reminders

- Role-based access control
-

Architecture

The system follows a traditional client-server architecture:



Technology Stack

Backend

- **Runtime:** Node.js
- **Framework:** Express.js
- **Database:** MongoDB with Mongoose ODM
- **Authentication:** JWT with cookies
- **File Upload:** Multer + Cloudinary
- **Scheduled Tasks:** node-cron
- **Environment Management:** dotenv
- **CORS:** cors middleware

Frontend

- **Framework:** React
 - **Routing:** React Router v6
 - **State Management:** Context API
 - **HTTP Client:** (Likely Axios/Fetch)
 - **UI Components:** Custom components
-

Project Structure

Backend Structure

```
backend/
├── src/
│   ├── controllers/           # Business logic
│   │   ├── accountManagementController.js
│   │   ├── braController.js
│   │   ├── clientController.js
│   │   ├── jobController.js
│   │   ├── kycController.js
│   │   ├── monthlyPaymentController.js
│   │   ├── notificationController.js
│   │   ├── operationController.js
│   │   ├── roleController.js
│   │   ├── serviceController.js
│   │   └── userController.js
│   ├── middleware/
│   │   ├── authMiddleware.js
│   │   ├── errorMiddleware.js
│   │   └── errorLoggingMiddleware.js
│   ├── models/               # Database schemas
│   ├── routes/               # API endpoints
│   │   ├── accountManagementRoutes.js
│   │   ├── authRoute.js
│   │   ├── braRoutes.js
│   │   ├── clientRoutes.js
│   │   ├── jobRoutes.js
│   │   ├── kycRoutes.js
│   │   ├── monthlyPaymentRoutes.js
│   │   ├── notificationRoutes.js
│   │   ├── operationRoutes.js
│   │   ├── roleRoute.js
│   │   ├── serviceRoutes.js
│   │   └── userRoute.js
│   ├── services/
│   │   └── fileUploadService.js
│   └── temp-uploads/         # Temporary file storage
├── .env                     # Environment variables
├── package.json
└── server.js                # Entry point
```

Frontend Structure

```
frontend/
├── src/
│   ├── components/
│   │   └── Layout.js
│   ├── context/
│   │   └── AuthContext.js
│   ├── pages/
│   │   ├── AccountManagement/
│   │   ├── AdminService/
│   │   ├── Auth/
│   │   ├── ClientProfile/
│   │   ├── Dashboard/
│   │   ├── Job/
│   │   │   ├── Admin/
│   │   │   ├── BRAManagement/
│   │   │   ├── ComplianceManagement/
│   │   │   ├── KYCManagement/
│   │   │   └── OperationManagement/
│   │   ├── Notifications/
│   │   └── UserManagement/
│   └── App.js
```

Core Features

1. User Management

- User registration and authentication
- Role-based permissions system
- Password reset functionality
- User profile management

2. Job Management

- Create jobs with client information
- Multi-stage approval workflow:
 - Admin creates job
 - Compliance reviews and approves/rejects
 - Operation management processes
 - KYC verification

- BRA approval
- Job status tracking and timeline
- Document attachment support

3. Client Management

- Client profile with comprehensive details
- Company information management
- Person details (Directors, Shareholders, Secretary, SEF)
- Engagement letter tracking
- Cross-job data synchronization

4. Document Management

- Support for multiple file types (PDF, Word, Images)
- Cloudinary integration for secure storage
- Document categorization (KYC, BRA, Company docs, etc.)
- File size limits and validation

5. Payment & Accounting

- Monthly payment tracking
- Invoice management
- Payment status updates
- Financial reporting

6. Notification System

- Real-time notifications for workflow updates
- Daily expiry notifications (via cron job)
- Unread notification tracking
- Email notifications for critical events

API Documentation

Base URL

- Development: `http://localhost:5000/api`

- Production: <https://testapp.newoon.com/api>

Authentication

All protected routes require JWT token in cookies.

Main API Endpoints

Authentication

- [POST /api/auth/register](#) - Register new user
- [POST /api/auth/login](#) - User login
- [GET /api/auth/logout](#) - User logout

Users

- [GET /api/users](#) - Get all users (requires userManagement permission)
- [GET /api/users/me](#) - Get current user
- [PUT /api/users/me](#) - Update current user
- [PUT /api/users/me/password](#) - Change password
- [GET /api/users/operation-managers](#) - Get operation managers list

Jobs

- [POST /api/jobs](#) - Create new job (admin only)
- [GET /api/jobs](#) - Get all jobs
- [GET /api/jobs/:id](#) - Get specific job
- [PUT /api/jobs/:id/update](#) - Update job
- [PUT /api/jobs/:id/approve](#) - Approve job
- [PUT /api/jobs/:id/reject](#) - Reject job
- [PUT /api/jobs/:id/cancel](#) - Cancel job
- [GET /api/jobs/:id/timeline](#) - Get job timeline
- [GET /api/jobs/check-job-number/:jobNumber](#) - Check job number availability

Operations

- [GET /api/operations/jobs/:jobId/company-details](#) - Get company details
- [PUT /api/operations/jobs/:jobId/company-details](#) - Update company details

- `GET /api/operations/jobs/:jobId/person-details/:personType` - Get person details
- `POST /api/operations/jobs/:jobId/person-details/:personType` - Add person
- `PUT /api/operations/jobs/:jobId/complete` - Complete operation
- `POST /api/operations/pre-approved-job` - Create pre-approved job

KYC Management

- `POST /api/kyc/jobs/:jobId/initialize` - Initialize KYC process
- `GET /api/kyc/jobs/:jobId/status` - Get KYC status
- `PUT /api/kyc/jobs/:jobId/lmro-approve` - LMRO approval
- `PUT /api/kyc/jobs/:jobId/dlmro-approve` - DLMRO approval
- `PUT /api/kyc/jobs/:jobId/ceo-approve` - CEO approval
- `PUT /api/kyc/jobs/:jobId/reject` - Reject KYC

BRA Management

- Similar structure to KYC with BRA-specific endpoints

Payments

- `POST /api/monthlypayment/add` - Add payment record
 - `GET /api/monthlypayment/history/:jobId` - Get payment history
 - `GET /api/monthlypayment/stats` - Get payment statistics
 - `PUT /api/monthlypayment/:id` - Update payment
 - `DELETE /api/monthlypayment/:id` - Delete payment
-

Authentication & Authorization

Authentication Flow

1. User logs in with credentials
2. Server validates and generates JWT token
3. Token stored in HTTP-only cookie
4. All subsequent requests include cookie

Permission System

The system uses a granular permission model:

javascript

```
permissions: {  
  userManagement: Boolean,  
  complianceManagement: Boolean,  
  operationManagement: Boolean,  
  kycManagement: {  
    lmro: Boolean,  
    dlmro: Boolean,  
    ceo: Boolean  
  },  
  braManagement: {  
    lmro: Boolean,  
    dlmro: Boolean,  
    ceo: Boolean  
  },  
  accountManagement: Boolean,  
  serviceManagement: Boolean,  
  admin: Boolean  
}
```

Middleware

- `protect` - Verifies JWT token
 - `adminOnly` - Checks for admin role
 - `checkPermission(permission)` - Validates specific permissions
-

Database Schema

Core Collections

Users

javascript

```
{  
  name: String,  
  email: String (unique),  
  password: String (hashed),  
  role: ObjectId (ref: 'Role'),  
  permissions: Object,  
  isActive: Boolean,  
  createdAt: Date  
}
```

Jobs

javascript

```
{
  jobNumber: String (unique),
  client: {
    name: String,
    email: String,
    phone: String,
    address: String
  },
  service: ObjectId (ref: 'Service'),
  status: String (enum: ['pending', 'approved', 'rejected', etc.]),
  assignedTo: ObjectId (ref: 'User'),
  documents: {
    passport: String,
    id: String,
    others: [String]
  },
  timeline: [{
    action: String,
    timestamp: Date,
    user: ObjectId,
    notes: String
  }],
  companyDetails: Object,
  personDetails: {
    directors: [Object],
    shareholders: [Object],
    secretary: [Object],
    sef: [Object]
  },
  kycStatus: Object,
  braStatus: Object,
  expiryDate: Date
}
```

MonthlyPayments

javascript

```
{
  job: ObjectId (ref: 'Job'),
  amount: Number,
  paymentDate: Date,
  status: String,
  invoiceFiles: [String],
  notes: String,
  createdBy: ObjectId (ref: 'User')
}
```

Notifications

javascript

```
{
  recipient: ObjectId (ref: 'User'),
  type: String,
  title: String,
  message: String,
  relatedJob: ObjectId,
  isRead: Boolean,
  createdAt: Date
}
```

File Handling

Upload Process

1. Files uploaded via Multer to temp directory
2. Validated for type and size
3. Uploaded to Cloudinary
4. Cloudinary URL stored in database
5. Temp file deleted

Supported File Types

- Images: JPEG, PNG
- Documents: PDF, DOC, DOCX
- Spreadsheets: XLS, XLSX

File Size Limits

- General uploads: 50MB
 - Large files (CEO approval): 100MB
 - Payment documents: 5MB
-

Frontend Structure

Routing

The application uses React Router v6 with nested routes:

```
javascript
```

```
<Route path="/" element={<Layout />}>
  <Route path="dashboard" element={<Dashboard />} />
  <Route path="create-job" element={<CreateJob />} />
  <Route path="compliance" element={<ComplianceManagement />} />
  <Route path="operation-management" element={<OperationManagement />} />
  <Route path="job/:jobId" element={<JobDetails />} />
  // ... more routes
</Route>
```

State Management

- **AuthContext:** Manages user authentication state
- Local component state for form handling
- API calls likely handled through custom hooks

Key Components

- **Layout:** Main application wrapper with navigation
 - **Dashboard:** Overview and statistics
 - **Job Components:** Create, view, and manage jobs
 - **Client Profile:** Comprehensive client information view
 - **User Management:** Admin user control panel
-

Setup & Installation

Prerequisites

- Node.js (v14 or higher)
- MongoDB (v4.4 or higher)
- Cloudinary account

Backend Setup

```
bash

# Navigate to backend directory
cd backend

# Install dependencies
npm install

# Create .env file with required variables
cp .env.example .env

# Start development server
npm run dev
```

Frontend Setup

```
bash

# Navigate to frontend directory
cd frontend

# Install dependencies
npm install

# Start development server
npm start
```

Database Setup

1. Install MongoDB locally or use MongoDB Atlas
 2. Create database named 'bms_db'
 3. Run seed scripts if available
-

Key Workflows

Job Creation Flow

1. **Admin creates job** → Status: 'pending'
2. **Compliance reviews** → Approve/Reject → Status: 'approved'/'rejected'
3. **Operation Management processes** → Updates company/person details
4. **KYC verification** → Multi-level approval (LMRO → DLMRO → CEO)
5. **BRA approval** → Multi-level approval (LMRO → DLMRO → CEO)
6. **Operation completes** → Status: 'operationCompleted'
7. **Payment processing** → Account management handles invoicing

Document Upload Flow

1. User selects file(s)
2. Frontend validates file type/size
3. Uploads to backend via multipart/form-data
4. Backend stores temporarily
5. Uploads to Cloudinary
6. Stores URL in database
7. Deletes temp file

Notification Flow

1. Action triggers notification (e.g., job approval)
 2. Notification created in database
 3. Real-time update to recipient
 4. Daily cron job for expiry notifications
-

Environment Variables

Required Backend Variables

```
env
```

```
# Server
```

```
PORT=5000
```

```
NODE_ENV=development
```

```
# Database
```

```
MONGO_URI=mongodb://localhost:27017/bms_db
```

```
# JWT
```

```
JWT_SECRET=your_jwt_secret_key
```

```
JWT_EXPIRE=30d
```

```
# Cloudinary
```

```
CLOUDINARY_CLOUD_NAME=your_cloud_name
```

```
CLOUDINARY_API_KEY=your_api_key
```

```
CLOUDINARY_API_SECRET=your_api_secret
```

```
# Email (if implemented)
```

```
EMAIL_HOST=smtp.gmail.com
```

```
EMAIL_PORT=587
```

```
EMAIL_USER=your_email
```

```
EMAIL_PASS=your_password
```

Common Issues & Solutions

Issue: CORS errors

Solution: Ensure frontend URL is in allowed origins list in backend

Issue: File upload fails

Solution:

- Check file size limits
- Verify Cloudinary credentials
- Ensure temp-uploads directory exists

Issue: Authentication fails

Solution:

- Check JWT secret matches

- Verify cookie settings for production
- Ensure HTTPS in production

Issue: MongoDB connection timeout

Solution:

- Check MongoDB service is running
- Verify connection string
- Check network/firewall settings

Issue: Cron jobs not running

Solution:

- Verify node-cron is installed
 - Check server timezone settings
 - Review cron expression syntax
-

Development Best Practices

1. Code Organization

- Keep controllers lean, business logic in services
- Use consistent naming conventions
- Comment complex business logic

2. Security

- Always validate user input
- Use parameterized queries
- Implement rate limiting
- Keep dependencies updated

3. Performance

- Implement pagination for large datasets
- Use database indexes appropriately
- Cache frequently accessed data
- Optimize file uploads

4. Testing

- Write unit tests for critical functions

- Test API endpoints with Postman/Insomnia
- Implement integration tests
- Test file upload edge cases

5. Version Control

- Use meaningful commit messages
 - Create feature branches
 - Review code before merging
 - Tag releases appropriately
-

Future Enhancements

Based on the current structure, potential improvements include:

1. **Email Integration:** Automated email notifications
 2. **Reporting Module:** Advanced analytics and reports
 3. **Mobile App:** React Native companion app
 4. **Webhook Support:** Third-party integrations
 5. **Audit Logging:** Comprehensive activity tracking
 6. **Backup System:** Automated database backups
 7. **API Rate Limiting:** Prevent abuse
 8. **WebSocket Integration:** Real-time updates
-

Support & Maintenance

For questions or issues:

1. Check this documentation
2. Review error logs
3. Check MongoDB logs
4. Verify environment variables
5. Contact system administrator

Remember to keep the system updated and monitor for security patches!